This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Breaking the Low-Rank Dilemma of Linear Attention

Qihang Fan[1,2], Huaibo Huang[1*], Ran He[1,2]
[1]MAIS & CRIPAC, Institute of Automation, Chinese Academy of Sciences, Beijing, China
[2]School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing, China
fanqihang.159@gmail.com, huaibo.huang@cripac.ia.ac.cn,
rhe@nlpr.ia.ac.cn

Abstract

The Softmax attention mechanism in Transformer models is notoriously computationally expensive due to its quadratic complexity, posing significant challenges in vision applications. In contrast, linear attention offers a far more efficient solution by reducing the complexity to linear levels. However, linear attention often suffers significant performance degradation compared to Softmax attention. Our experiments indicate that this performance drop stems from the low-rank nature of linear attention's output feature map, which hinders its ability to adequately model complex spatial information. To address this low-rank dilemma, we conduct rank analysis from two perspectives: the KV buffer and the output features. Consequently, we introduce ***Rank-Augmented Linear Attention*** *(RALA), which rivals the performance of Softmax attention while maintaining linear complexity and high efficiency. Building upon RALA, we construct the* ***Rank-Augmented Vision Linear Transformer*** *(RAVLT). Extensive experiments demonstrate that RAVLT achieves excellent performance across various vision tasks. Specifically, without using any additional labels, data, or supervision during training, RAVLT achieves an* ***84.4%*** *Top-1 accuracy on ImageNet-1k with only* ***26M*** *parameters and* ***4.6G*** *FLOPs. This result significantly surpasses previous linear attention mechanisms, fully illustrating the potential of RALA. Code will be available at* https://github.com/qhfan/RALA.

1. Introduction

In recent years, the Transformer [55] has garnered increasing attention as a powerful foundational architecture. In the field of computer vision, Vision Transformers [12] have made significant breakthroughs in image classification, object detection, instance segmentation, and semantic segmentation, further demonstrating the vast potential of Transformers.

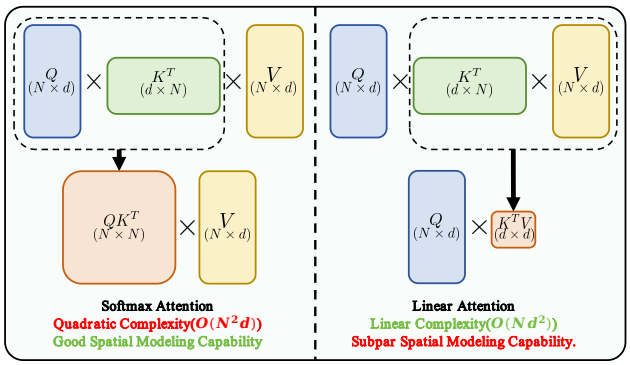

Figure 1. Comparison of Softmax attention and linear attention. Linear attention has linear complexity and high efficiency, but its spatial modeling capability is inferior to Softmax attention.

However, applying Transformers to vision tasks is not straightforward. Self-attention, as the core mechanism of Transformers, has quadratic complexity. As the number of tokens increases, the computational load of self-attention grows significantly, making Transformers challenging to apply to vision tasks. Many works attempt to address this issue, with some approaches grouping vision tokens to limit the number of tokens each token can interact with [11, 16, 42, 43, 54, 58, 69]. While this method effectively reduces the model's complexity, it sacrifices the Transformer's crucial ability to perceive information globally. Another line of work attempts to downsample the keys and values in Softmax attention to capture global features with lower complexity [15, 26, 56, 57]. However, this approach sacrifices the model's fine-grained perception capabilities.

Unlike the methods mentioned above, linear attention [2, 23, 24, 32, 45, 50] takes a different approach by replacing the Softmax with kernel functions and altering the computation order of Q, K, and V, which is shown in the Fig. 1. This reduces the quadratic complexity of Softmax attention to linear complexity. Additionally, because its computation closely resembles Softmax attention and only uses kernel functions to approximate Softmax, linear attention pos-

*Huaibo Huang is the corresponding author.

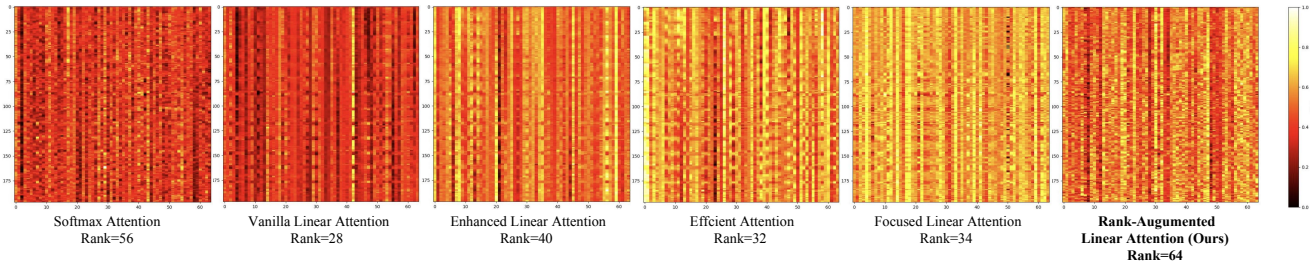

Figure 2. Comparison of feature maps output by Softmax attention and different linear attentions. All experiments are conducted based on the DeiT-T architecture, with $N = 196$ and $d = 64$. The full rank of matrices in the fig is 64. Compared to Softmax attention, the output features of various linear attentions exhibit significantly low-rank properties. This indicates that the diversity of features learned by linear attention is inferior to that learned by Softmax attention.

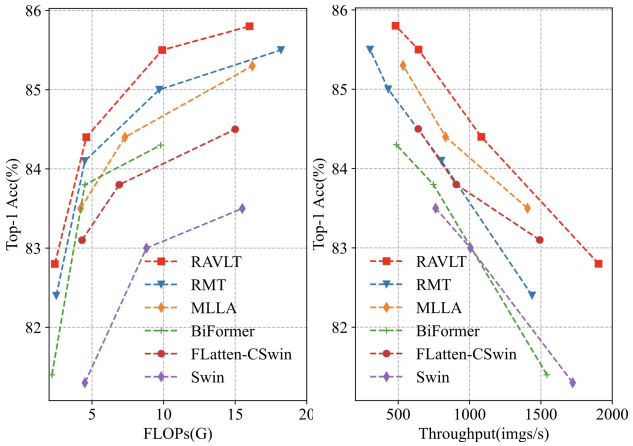

Figure 3. Comparison among models based on linear attention and Softmax attention. Our RAVLT achieves state-of-the-art results across all scales and significantly outperforms existing vision models based on linear attention.

sesses excellent global modeling capability. Furthermore, since linear attention does not involve downsampling, it retains the model's fine-grained perception ability.

Despite having a form and advantages very similar to Softmax attention, the actual performance of linear attention is less than satisfactory. Although it is highly efficient, its performance shows a significant gap compared to Softmax attention. To identify the cause of this performance gap, we conduct a comparison based on the DeiT-T architecture, merely replacing different attention mechanisms (vanilla linear attention, Enhanced Linear Attention [2], Efficient Attention [50], Focused Linear Attention [23]), with results shown in Fig. 2. We perform rank analysis on the features of single heads from different output feature maps ($N = 196$, $d = 64$, with the full rank of the matrix being 64). We find that the rank of output features from existing linear attentions are significantly lower than those from Softmax attention, indicating that the diversity of features they learn is comparatively poorer.

Based on the above considerations, we aim to increase the rank of linear attention to achieve a trade-off between performance and efficiency in vision models. Specifically, we find that two computational steps in linear attention—the calculation of the KV buffer and the calculation of the output features—both affect its low-rank characteristics. Based on this, we propose **Rank-Augmented Linear Attention** (RALA). In RALA, to address the low-rank issue of the KV buffer, we use a set of context-aware rearrangement coefficients to restructure the weights of each token in the KV buffer. This approach enhances the richness and diversity of information in the KV buffer, which in turn increases the rank of the matrix. For the low-rank issue of the output features, we introduce a feature interaction strategy in the channel dimension, ensuring the output features reach a full-rank state. RALA effectively enhances the rank of the feature matrices in linear attention, enabling it to model complex spatial features with linear complexity. As shown in the Fig. 2, the output feature matrix of RALA reaches the full rank of 64.

Utilizing RALA, we construct the **Rank-Augmented Vision Linear Transformer** (RAVLT). We conduct extensive experiments with RAVLT on image classification, object detection, instance segmentation, and semantic segmentation, and RAVLT achieves results comparable to state-of-the-art Vision Transformers. RAVLT also exhibits great trade-off between performance and efficiency. As shown in Fig. 3, without using any additional training data, labels, or supervision, our model achieves an accuracy of **84.4%** on ImageNet with only **26M** parameters and **4.6G** FLOPs. As far as we konw, this surpasses all existing vision models.

2. Related works

Vision Transformer. Transformer is first employed in the field of NLP [55] and have attracted wide interest in computer vision community [12, 18, 60, 68]. The core operation of Transformer is the vanilla Softmax attention, which faces the challenge of quadratic complexity. Many methods have been proposed to address this challenge [10, 14, 17, 21, 25, 42, 49, 61]. Some methods group tokens to limit the receptive field of each token, thereby reducing the complexity of vanilla Softmax attention to linear [11, 16, 30, 42, 43, 52, 54, 58, 65]. Another approach downsamples the key and value, sacri-

ficing some fine-grained perception while preserving the Transformer's ability to perceive global information, and reduces the complexity of vanilla Softmax attention to linear [4, 15, 21, 56, 57, 61]. Many methods also try to combine classical CNNs with Transformers, leveraging the excellent local perception capabilities of CNNs with the global perception abilities of Transformers to create a powerful vision backbone [8, 14, 21, 22, 28, 33, 38, 47, 51, 66]. All these models rely on the vanilla Softmax attention, which incurs quadratic complexity.

Linear Attention. Linear attention uses kernel functions to approximate the Softmax function, allowing it to change the computation order of Q, K, and V in vanilla Softmax attention, thereby reducing the computational complexity from quadratic to linear. Although the efficiency of linear attention is significantly improved compared to vanilla Softmax attention, its performance still lags considerably behind. Many methods have been proposed to improve this [23, 24, 45, 50, 63, 67]. Specifically, FLatten-Transformer [23] proposes focused function to improve the focus ability of linear attention. MLLA [24] connects the linear attention with Mamba [19]. Most of other methods start by using kernel functions to approximate Softmax or introduce DWConv to increase feature diversity. However, the low-rank issue still exists, as shown in Fig. 2.

3. Method

3.1. Preliminary

Softmax Attention. Softmax attention is the classic mechanism used in Transformers [55] and employed in ViT and its various variants [11, 12, 17, 31, 33, 43]. Specifically, given the input tokens $X \in \mathbb{R}^{N\times d}$, Softmax attention can generally be represented as:

$$Y_i = \sum_{j=1}^{N} \frac{\mathrm{Sim}(Q(X_i), K(X_j))}{\sum_{m=1}^{N} \mathrm{Sim}(Q(X_i), K(X_m))} V(X_j) \quad (1)$$

where i, j are the index of tokens, $X_i, Y_i \in \mathbb{R}^{1\times d}$. $Q(.)$, $K(.)$ and $V(.)$ are the linear transformations for X. For simplicity, we use Q_i to refer to $Q(X_i)$, and similarly for $K(.)$ and $V(.)$. In Softmax attention, it is generally expressed as: $\mathrm{Sim}(Q_i, K_j) = \exp(Q_i K_j^T/\sqrt{d})$. Since the computation of Softmax attention must be performed between all queries and keys, its complexity is $O(N^2d)$, exhibiting quadratic growth as the number of tokens increases. Therefore, applying it directly in vision models results in significant computational overhead, especially when the number of tokens is large.

Linear Attention. Compared to Softmax attention, linear attention uses kernel functions to approximate the $\mathrm{Sim}(.,.)$, allowing the computation of $\mathrm{Sim}(Q_i, K_j)$ to be decomposed into $\mathrm{Sim}(Q_i, K_j) = \kappa(Q_i)\kappa(K_j)^T$. Specifically, the computation of linear attention is shown as follows:

$$\begin{aligned} Y_i &= \sum_{j=1}^{N} \frac{\mathrm{Sim}(Q_i, K_j)}{\sum_{m=1}^{N} \mathrm{Sim}(Q_i, K_m)} V_j \\ &= \sum_{j=1}^{N} \frac{\kappa(Q_i)\kappa(K_j)^T}{\sum_{m=1}^{N} \kappa(Q_i)\kappa(K_m)^T} V_j \\ &= \frac{\kappa(Q_i)(\sum_{j=1}^{N} \kappa(K_j)^T V_j)}{\kappa(Q_i)(\sum_{m=1}^{N} \kappa(K_m)^T)} \end{aligned} \quad (2)$$

where $\kappa(.)$ is the kernel function. Compared to the computation order $(QK^T)V$ in Softmax attention, the computation order $Q(K^TV)$ in linear attention reduces the complexity with respect to the number of tokens (N) from quadratic $O(N^2d)$ to linear $O(Nd^2)$. However, the reduction in complexity also results in a decline in performance. As shown in Fig. 2, the output features of linear attention have a low rank, indicating that linear attention fails to learn diverse visual features.

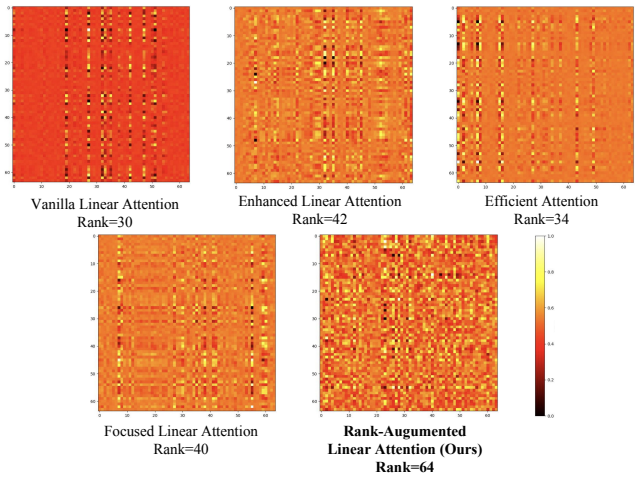

Figure 4. Visualization of the rank analysis of the KV buffer for different linear attention mechanisms. The KV buffer $\sum_{j=1}^{N} \kappa(K_j)^T V_j \in \mathbb{R}^{64\times 64}$.

3.2. Rank-Augmented Linear Attention

In the general expression of linear attention, since the denominator $\kappa(Q_i)(\sum_{j=1}^{N} \kappa(K_j)^T) \in \mathbb{R}^{1\times 1}$, the rank of the output feature matrix Y only depends on the numerator. Therefore, in the subsequent analysis, we focus solely on the numerator $\kappa(Q_i)(\sum_{j=1}^{N} \kappa(K_j)^T V_j)$.

Augment the rank of the KV buffer. Each Q_i needs to be multiplied by the KV buffer $(\sum_{j=1}^{N} \kappa(K_j)^T V_j)$ to obtain the final Y_i. Therefore, the diversity of the features in the

KV buffer, reflected by its rank, dictates the diversity of the final output features. Thus, we first analyze the rank of the KV buffer. As shown in Fig. 4, the rank of the KV buffer is relatively low in almost all linear attentions. This means that the KV buffer does not store sufficient information for each Q_i to query. The feature diversity in the KV buffer is poor.

To increase the rank of the KV buffer, we introduce operations into its calculation to eliminate linear dependencies between different rows and columns. Beyond the feature diversity, the rank of the KV buffer is also closely related to the amount of information it contains, increasing the information content in the KV buffer will simultaneously raise its rank. Based on this consideration, rather than assigning a weight of 1 to each $\kappa(K_j)^T V_j, j \in [1, N]$ as in other linear attentions [23, 50], we aim to highlight the significance of each token in the KV buffer by assigning higher weights to the tokens that contain more information. That is:

$$B = \sum_{j=1}^{N} \alpha_j \kappa(K_j)^T V_j, \quad \sum_{j=1}^{N} \alpha_j = N \tag{3}$$

where B denotes the KV buffer, α_j is the weight coefficient of the j-th token.

To determine the value of α_j, we draw inspiration from the design of Softmax attention. In Softmax attention, the attention scores between tokens are generally considered to represent the degree of correlation between them. Tokens more correlated with the global token are considered to contain more important information[39]. Based on this fact, by calculating the attention score of the global query to each key, we determine the proportion of each token in the KV buffer, resulting in the following:

$$\begin{aligned} Q_g &= \frac{\sum_{i=1}^{N} Q_i}{N} \\ \alpha_j &= N \times \frac{\exp(Q_g \kappa(K_j)^T)}{\sum_{m=1}^{N} \exp(Q_g \kappa(K_m)^T)} \end{aligned} \tag{4}$$

As shown in Fig. 4, after introducing α_j as the modulation coefficient for $\kappa(K_j)^T V_j$, the information content in the KV buffer increases, which significantly enhances its rank. More analysis about why the introduction of α_j can increase the matrix's rank can be found in Appendix.

It should be emphasized that the summation of matrices can indeed increase their rank. This is fundamentally different from a simple linear combination, which cannot increase the rank. For example:

$$1 \times \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix} + 1 \times \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix}$$

A linear combination of two rank-1 matrices can result in a matrix with rank 3.

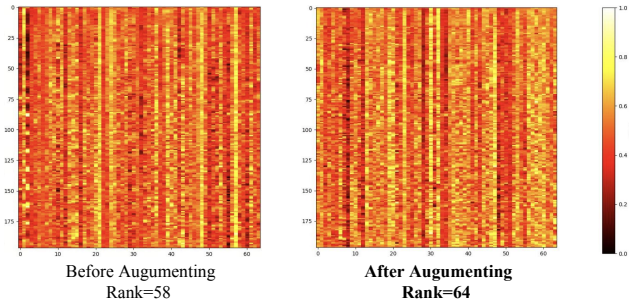

Figure 5. Visualization of the output features' ($Y \in \mathbb{R}^{N \times d}$, $N = 196$, $d = 64$) rank analysis.

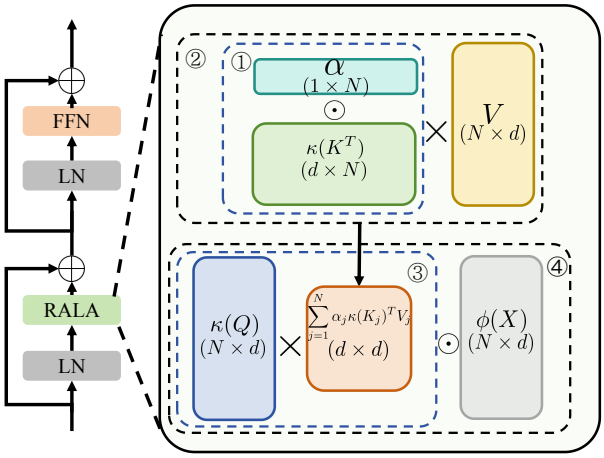

Figure 6. RAVLT's block (left) and RALA (right). We employ serial numbers to represent the sequence of calculations. $\odot$ represents Hadamard product, and $\times$ represents matrix multiplication.

Augment the rank of the output features. In the fundamentals of matrix theory, for matrices C and D, we have: $\mathrm{Rank}(CD) \leq \min(\mathrm{Rank}(C), \mathrm{Rank}(D))$. Therefore, when calculating the output features $Y = \kappa(Q)B$, its rank:

$$\begin{gathered} \mathrm{Rank}(Y) \leq \min(\mathrm{Rank}(\kappa(Q)), \mathrm{Rank}(B)) \leq d \\ Y \in \mathbb{R}^{N \times d}, \ \kappa(Q) \in \mathbb{R}^{N \times d}, \ B = \sum_{j=1}^{N} \alpha_j \kappa(K_j)^T V_j \in \mathbb{R}^{d \times d} \end{gathered} \tag{5}$$

This indicates that even though the KV buffer B reaches a full-rank state, the output feature matrix is still highly likely to be in a low-rank state ($\mathrm{Rank}(Y) < d$), meaning information loss occurs when computing each Y_i. As shown in Fig. 5, we confirm this point when performing rank analysis on the output features. The output features exhibit a certain degree of rank reduction. To restore the output feature matrix Y to a full-rank state, we aim to complete the information loss for each Y_i.

Specifically, we post-process each output feature Y_i by modulating Y_i with a transformed form that contains the information of the original token X_i. At this stage, Y_i has already completed token mixing, so this modulation is ap-

Model	Blocks	Channels	Heads	Params (M)	FLOPs (G)
RAVLT-T	[2,2,6,2]	[64,128,256,512]	[1,2,4,8]	15	2.4
RAVLT-S	[3,5,9,3]	[64,128,320,512]	[1,2,5,8]	26	4.6
RAVLT-B	[4,6,12,6]	[96,192,384,512]	[1,2,6,8]	48	9.9
RAVLT-L	[4,7,19,8]	[96,192,448,640]	[1,2,7,10]	95	16.0

Table 1. Architecture variants of RAVLT. The FLOPs are measured at resolution 224×224.

plied only along the channel dimension. Consequently, each output feature becomes:

$$Y_i = \phi(X_i) \odot (\kappa(Q_i) \sum_{j=1}^{N} \alpha_j \kappa(K_j)^T V_j) \tag{6}$$

where $\odot$ denotes the Hadamard product. As shown in the Fig. 5, After introducing the modulation coefficients, the feature diversity of the output feature matrix is significantly enhanced, transforming it from a low-rank state to a full-rank state. This indicates that token-specific modulation adds a degree of feature diversity to the output features. More analysis can be found in the Appendix.

Design of the block. Fig. 6 illustrates the design of the entire block. Our block design follows the classic Transformer block design [55], consisting of linear attention and FFN. All the networks in this paper are constructed by stacking this basic block. We use a simple linear projection to serve as the transformation function $\phi(.)$. As for $\kappa(.)$, we have $\kappa(.) = \mathrm{Elu}(.) + 1$. For each block, we use CPE [6] to serve as the positional encoding. The CPE is a simple 3×3 depth-wise convolution with residual connection.

3.3. Implementation Details

Based on the block in the Fig. 6, we construct the **Rank-Augmented Vision Linear Transformer** (RAVLT), a hierarchical general vision backbone. As shown in the Tab. 1, following the previous works [17, 31, 69], we build three RAVLT backbones with different settings of block number and channel number in each stage. In each stage, the image downsampling rates are $\frac{1}{4}$, $\frac{1}{8}$, $\frac{1}{16}$, and $\frac{1}{32}$, respectively. At the beginning of each stage, we use a 3×3 convolution with the stride of 2 to downsample the image.

4. Experiments

We conduct extensive experiments on multiple vision tasks, such as image classification, object detection, instance segmentation, and semantic segmentation. We also make ablation studies to validate the importance of each components in RALA. More details can be found in Appendix.

4.1. Image Classification

Settings. We train our models on ImageNet-1K [9] from scratch and follow the same training strategy in DeiT [53] with the only supervision being classification loss. The maximum rates of increasing stochastic depth [29] are set to 0.1/0.15/0.4/0.55 for RAVLT-T/S/B/L, respectively. We evaluate the models on ImageNet [9] and ImageNet-V2 [48].

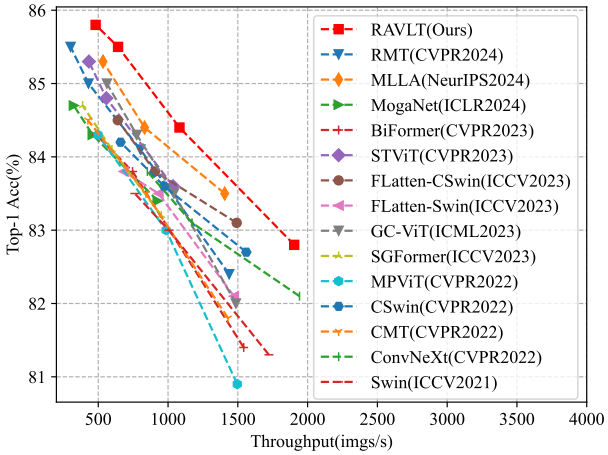

Figure 7. Comparison of multiple general backbones' inference speed. The speed are measured on A100. RAVLT exhibits the best trade-off between efficiency and performance.

Cost	Model	Params(M)	FLOPs(G)	Top1-acc(%)
Tiny model ~ 2.5G	tiny-MOAT-2 [64]	10	2.3	70.1
	BiFormer-T [69]	13	2.2	70.7
	SMT-T [41]	12	2.4	71.0
	RMT-T [17]	14	2.5	72.1
	RAVLT-T	15	2.4	**72.7**
Small model ~ 4.5G	DeiT-S	22	4.6	68.5
	MOAT-0 [64]	28	5.7	72.8
	SMT-S [41]	21	4.7	73.3
	MLLA-T [24]	25	4.2	73.3
	BiFormer-S [69]	26	4.5	73.6
	RMT-S [17]	27	4.5	74.1
	RAVLT-S	26	4.6	**74.9**
Base model ~ 10.0G	XCiT-S24 [13]	48	9.1	73.3
	BiFormer-B [69]	57	9.8	74.0
	MOAT-1 [64]	42	9.1	74.2
	MLLA-S [24]	43	7.3	74.9
	RMT-B [17]	54	9.7	75.6
	RAVLT-B	48	9.9	**76.3**
Large model ~ 15.0G	DeiT-B [53]	86	17.5	71.5
	MOAT-2 [64]	73	17.2	74.3
	RMT-L [17]	95	18.2	76.3
	MLLA-B [24]	96	16.2	76.7
	RAVLT-L	95	16.0	**76.8**

Table 2. Classification results of different general backbones on ImageNet-V2

Results. In Tab. 3 and Tab. 2, we compare the performance of various models on ImageNet and ImageNet-v2, respectively. Under models of comparable size, RAVLT achieves the best results. Specifically, RAVLT-B surpasses RMT-B by **0.5%** with the same FLOPs. This result demonstrates the superior of RAVLT.

Cost	Model	Type	Parmas (M)	FLOPs (G)	Top1-acc (%)
Tiny model ~ 2.5G	VAN-B1 [22]	CNN	14	2.5	81.1
	Conv2Former-N [28]	CNN	15	2.2	81.5
	MPViT-XS [35]	Trans	11	2.9	80.9
	NAT-M [25]	Trans	20	2.7	81.8
	GC-ViT-XT [26]	Trans	20	2.6	82.0
	SMT-T [41]	Trans	12	2.4	82.2
	RMT-T [17]	Trans	14	2.5	82.4
	FL-PVT-T [23]	Linear	12	2.0	77.8
	SOFT-Tiny [45]	Linear	13	1.9	79.3
	FL-PVTv2-B1 [23]	Linear	13	2.2	79.5
	RAVLT-T	Linear	15	2.4	**82.8**
Small model ~ 4.5G	ConvNeXT-T [44]	CNN	29	4.5	82.1
	MogaNet-S [37]	CNN	25	5.0	83.4
	InternImage-T [59]	CNN	30	5.0	83.5
	Swin-T [43]	Trans	29	4.5	81.3
	StructViT-S-8-1 [33]	Trans	24	5.4	83.3
	MOAT-0 [64]	Trans	28	5.7	83.3
	FAT-B3 [15]	Trans	29	4.4	83.6
	SMT-S [41]	Trans	20	4.8	83.7
	BiFormer-S [69]	Trans	26	4.5	83.8
	RMT-S [17]	Trans	27	4.5	84.1
	FL-Swin-T [23]	Linear	29	4.5	82.1
	SOFT-Small [45]	Linear	24	3.3	82.2
	FL-CSwin-T [23]	Linear	21	4.3	83.1
	MLLA-T [24]	Linear	25	4.2	83.5
	RAVLT-S	Linear	26	4.6	**84.4**
Base model ~ 10.0G	ConvNeXT-S [44]	CNN	50	8.7	83.1
	InternImage-S [59]	CNN	50	8.0	84.2
	DaViT-S [10]	Trans	50	8.8	84.2
	StructViT-B-8-1 [33]	Trans	52	12.0	84.3
	BiFormer-B [69]	Trans	57	9.8	84.3
	GC-ViT-S [26]	Trans	51	8.5	84.3
	STViT-B [31]	Trans	52	9.9	84.8
	RMT-B [17]	Trans	54	9.7	85.0
	SOFT-Large [45]	Linear	64	11.0	83.1
	FL-Swin-S [23]	Linear	51	8.7	83.5
	FL-CSwin-S [23]	Linear	35	6.9	83.8
	MLLA-S [24]	Linear	43	7.3	84.4
	RAVLT-B	Linear	48	9.9	**85.5**
Large model ~ 15.0G	HorNet-B [47]	CNN	88	15.5	84.3
	InterImage-B [59]	CNN	97	16.0	84.9
	SMT-L [41]	Trans	81	17.7	84.6
	SG-Former-B [20]	Trans	78	15.6	84.7
	GC-ViT-B [26]	Trans	90	14.8	85.0
	FAT-B5 [15]	Trans	88	15.1	85.2
	STViT-L [31]	Trans	95	15.6	85.3
	RMT-L [17]	Trans	95	18.2	85.5
	SOFT-Huge [45]	Linear	87	16.3	83.3
	FLatten-Swin-B [23]	Linear	89	15.4	83.8
	FLatten-CSwin-B [23]	Linear	75	15.0	84.5
	MLLA-B [24]	Linear	96	16.2	85.3
	RAVLT-L	Linear	95	16.0	**85.8**

Table 3. Comparison with the state-of-the-art on ImageNet-1K classification. We use "CNN" to refer to convolutional neural networks, "Trans" to refer to Vision Transformers, and "Linear" to refer to models based on linear operators.

Backbone	Type	Params (M)	FLOPs (G)	Mask R-CNN 3×+MS AP^b	AP^b_{50}	AP^b_{75}	AP^m	AP^m_{50}	AP^m_{75}
NAT-T [25]	Trans	48	258	47.8	69.0	52.6	42.6	66.0	45.9
SMT-S [41]	Trans	40	265	49.0	70.1	53.4	43.4	67.3	46.7
RMT-S [17]	Trans	46	262	50.7	71.9	55.6	44.9	69.1	48.4
FL-Swin-T [23]	Linear	49	268	46.5	68.5	50.8	42.1	65.4	45.1
MLLA-T [24]	Linear	44	255	48.8	71.0	53.6	43.8	68.0	46.8
RAVLT-S	Linear	44	262	**51.4**	**72.3**	**56.5**	**45.5**	**69.7**	**48.8**
InternImage-S [59]	CNN	69	340	49.7	71.1	54.5	44.5	68.5	47.8
NAT-S [25]	Trans	70	330	48.4	69.8	53.2	43.2	66.9	46.4
SMT-B [41]	Trans	52	328	49.8	71.0	54.4	44.0	68.0	47.3
RMT-B [17]	Trans	73	373	52.2	72.9	57.0	46.1	70.4	49.9
MLLA-S [24]	Linear	63	319	50.5	71.8	55.2	44.9	69.1	48.2
RAVLT-B	Linear	67	372	**52.7**	**73.5**	**57.7**	**46.4**	**70.6**	**50.2**
InternImage-B [59]	CNN	115	501	50.3	71.4	55.3	44.8	68.7	48.0
Swin-B [43]	Trans	107	496	48.6	70.0	53.4	43.3	67.1	46.7
CSwin-B [11]	Trans	97	526	50.8	72.1	55.8	44.9	69.1	48.3
RAVLT-L	Linear	114	501	**53.6**	**74.4**	**58.9**	**47.3**	**71.6**	**51.2**

Table 4. Comparison to other backbones using Mask R-CNN with "3 × +MS" schedule.

Backbone	Type	Params (M)	FLOPs (G)	Cascade Mask R-CNN 3×+MS AP^b	AP^b_{50}	AP^b_{75}	AP^m	AP^m_{50}	AP^m_{75}
HorNet-T [47]	CNN	80	728	52.4	71.6	56.8	45.6	69.1	49.6
SMT-S [41]	Trans	78	744	51.9	70.5	56.3	44.7	67.8	48.6
UniFormer-S [36]	Trans	79	747	52.1	71.1	56.6	45.2	68.3	48.9
RMT-S [17]	Trans	83	741	53.2	72.0	57.8	46.1	69.8	49.8
FL-Swin-T [23]	Linear	87	747	50.8	69.6	55.1	44.1	67.0	48.1
RAVLT-S	Linear	82	741	**54.2**	**72.9**	**58.7**	**46.8**	**70.5**	**50.9**
HorNet-S [47]	CNN	108	827	53.3	72.3	57.8	46.3	69.9	50.4
NAT-S [25]	Trans	108	809	51.9	70.4	56.2	44.9	68.2	48.6
GC-ViT-S [26]	Trans	108	866	52.4	71.0	57.1	45.4	68.5	49.3
DAT-S [61]	Trans	107	857	52.7	71.7	57.2	45.5	69.1	49.3
RMT-B [17]	Trans	111	852	54.5	72.8	59.0	47.2	70.5	51.4
RAVLT-B	Linear	105	851	**55.3**	**73.8**	**60.1**	**47.7**	**71.4**	**52.1**
ConvNeXt-B [44]	CNN	145	964	52.7	71.3	57.2	45.6	68.9	49.5
Swin-B [43]	Trans	145	982	51.9	70.9	56.5	45.0	68.4	48.7
CSwin-B [11]	Trans	135	1004	53.9	72.6	58.5	46.4	70.0	50.4
RAVLT-L	Linear	152	979	**55.6**	**74.1**	**60.5**	**48.0**	**71.8**	**52.3**

Table 5. Comparison to other backbones using Cascade Mask R-CNN with "3 × +MS" schedule.

4.2. Inference Speed

We evaluate the RAVLT's inference efficiency on A100, the results are shown in the Fig. 7. It can be seen that our models exhibit better trade-off between performance and efficiency than other models.

4.3. Object Detection and Instance Segmentation

Settings. We adopt MMDetection [5] to implement RetinaNet [40], Mask-RCNN [27] and Cascade Mask R-CNN [3]. Following previous works [17, 43, 56], we use the commonly used "1×" (12 training epochs) setting for the RetinaNet and Mask R-CNN. Besides, we use "3 × +MS" for Mask R-CNN and Cascade Mask R-CNN.

Results. We show the comparison results in the Tab. 4, Tab. 5, and Tab. 6. Compared to its competitors, RAVLT achieves excellent results across every detection framework. Specifically, with the framework of Cascade Mask R-CNN and "3×+MS" schedule, RAVLT-B achieves **55.3**AP^b and **47.7**AP^m. The result even surpasses CSwin-B, which is

Backbone	Type	Params (M)	FLOPs (G)	Mask R-CNN 1×						Params (M)	FLOPs (G)	RetinaNet 1×					
				AP^b	AP^b_{50}	AP^b_{75}	AP^m	AP^m_{50}	AP^m_{75}			AP^b	AP^b_{50}	AP^b_{75}	AP^b_S	AP^b_M	AP^b_L
PVTv2-B1 [57]	Trans	33	243	41.8	54.3	45.9	38.8	61.2	41.6	23	225	41.2	61.9	43.9	25.4	44.5	54.3
SOFT++-Tiny [46]	Linear	32	212	41.2	63.7	44.7	38.2	61.0	41.0	23	200	41.9	62.7	44.7	27.8	45.4	55.6
FL-PVT-T [23]	Linear	32	244	38.2	61.6	41.9	37.0	57.6	39.0	–	–	–	–	–	–	–	–
RAVLT-T	Linear	33	219	**47.3**	**69.1**	**51.9**	**42.7**	**66.2**	**46.0**	24	201	**45.9**	**67.4**	**49.4**	**28.5**	**50.1**	**60.8**
InternImage-T [59]	CNN	49	270	47.2	69.0	52.1	42.5	66.1	45.8	–	–	–	–	–	–	–	–
CMT-S [21]	Trans	45	249	44.6	66.8	48.9	40.7	63.9	43.4	44	231	44.3	65.5	47.5	27.1	48.3	59.1
RMT-S [17]	Trans	46	262	49.0	70.8	53.9	43.9	67.8	47.4	36	244	47.8	69.1	51.8	32.1	51.8	63.5
FL-Swin-T [23]	Linear	49	268	44.2	67.3	48.5	40.2	63.8	43.0	–	–	–	–	–	–	–	–
MLLA-T [24]	Linear	44	255	46.8	69.5	51.5	42.1	66.4	45.0	–	–	–	–	–	–	–	–
RAVLT-S	Linear	44	262	**49.8**	**71.3**	**54.5**	**44.6**	**68.5**	**48.2**	34	244	**48.3**	**69.8**	**52.1**	**32.7**	**52.8**	**63.6**
InternImage-S [59]	CNN	69	340	47.8	69.8	52.8	43.3	67.1	46.7	–	–	–	–	–	–	–	–
STViT-B [31]	Trans	70	359	49.7	71.7	54.7	44.8	68.9	48.7	–	–	–	–	–	–	–	–
SOFT++-medium [46]	Linear	69	342	46.6	67.8	51.2	42.0	64.8	45.2	59	322	44.3	64.7	47.4	29.0	48.2	59.9
MLLA-S [24]	Linear	63	319	49.2	71.5	53.9	44.2	68.5	47.2	–	–	–	–	–	–	–	–
RAVLT-B	Linear	67	372	**51.2**	**72.7**	**56.4**	**45.7**	**69.9**	**49.5**	57	353	**49.8**	**71.2**	**54.0**	**34.0**	**54.3**	**64.9**
InternImage-B [59]	CNN	115	501	48.8	70.9	54.0	44.0	67.8	47.4	–	–	–	–	–	–	–	–
Swin-B [43]	Trans	107	496	46.9	69.2	51.6	42.3	66.0	45.5	98	477	45.0	66.4	48.3	28.4	49.1	60.6
RMT-L [17]	Trans	114	557	51.6	73.1	56.5	45.9	70.3	49.8	104	537	49.4	70.6	53.1	34.2	53.9	65.2
MLLA-B [24]	Linear	115	502	50.5	72.0	55.4	45.0	69.3	48.6	–	–	–	–	–	–	–	–
RAVLT-L	Linear	114	501	**52.3**	**73.8**	**57.3**	**46.4**	**71.1**	**50.4**	104	482	**50.9**	**72.2**	**55.0**	**34.7**	**55.7**	**65.4**

Table 6. Comparison to other backbones using RetinaNet and Mask R-CNN with "1×" schedule.

Model	Type	Semantic FPN 80K			Upernet 160K		
		Params (M)	FLOPs (G)	mIoU (%)	Params (M)	FLOPs (G)	$mIoU_{ss}$ (%)
VAN-B1 [22]	CNN	18	140	42.9	–	–	–
PVTv2-B1 [57]	Trans	18	136	42.5	–	–	–
RMT-T [17]	Trans	17	136	46.4	–	–	–
FL-PVT-T [23]	Linear	16	169	37.2	–	–	–
RAVLT-T	Linear	18	136	**47.9**	44	893	**49.3**
MogaNet-S [37]	CNN	29	189	47.7	55	946	49.2
StructViT-S [33]	Trans	26	271	46.9	–	–	–
SMT-S [41]	Trans	–	–	–	50	935	49.2
SGFormer-S [20]	Trans	25	205	49.0	53	989	49.9
RMT-S [17]	Trans	30	180	49.4	56	937	49.8
Fl-Swin-T [23]	Linear	–	–	–	60	946	44.8
RAVLT-S	Linear	28	180	**49.5**	55	937	**50.7**
MogaNet-B [37]	CNN	–	–	–	74	1050	50.1
InterImage-S [59]	CNN	–	–	–	80	1017	50.2
DAT-S [61]	Trans	53	320	46.1	81	1079	48.3
StructViT-B [33]	Trans	54	529	48.5	–	–	–
RMT-B [17]	Trans	57	294	50.4	83	1051	52.0
FL-Swin-S [23]	Linear	–	–	–	82	1038	48.1
RAVLT-B	Linear	51	292	**51.9**	77	1050	**52.5**
MogaNet-L [37]	CNN	–	–	–	113	1176	50.9
CSWin-B [11]	Trans	81	464	49.9	109	1222	51.1
SGFormer-B [20]	Trans	81	475	50.6	109	1304	52.0
RMT-L [17]	Trans	98	482	51.4	125	1241	52.8
MLLA-B [24]	Linear	–	–	–	128	1183	51.9
RAVLT-L	Linear	98	424	**52.6**	125	1182	**53.2**

Table 7. Comparison with the state-of-the-art on ADE20K.

much larger than RAVLT-B.

4.4. Semantic Segmentation

Settings. We adopt the Semantic FPN [34] and UperNet [62] based on MMSegmentation [7], apply RAVLTs which are pretrained on ImageNet-1K as backbone. We use the same setting of PVT [56] to train the Semantic FPN, and follow the default settings in Swin [43] to train the UperNet.

Results. We show the segmentation results in the Tab. 7. RAVLT surpasses its competitors across various frameworks. For example, with the framework of Semantic FPN, RAVLT-B achieves **51.9**mIoU. The result even surpasses larger RMT-L. With the framework of UperNet, RAVLT can achieve the mIoU of **53.2**.

4.5. Comparison with Other Linear Attention

For a fair comparison with other linear attention mechanisms, similar to FLatten-Transformer [23], we use the DeiT-T [53], Swin-T, and Swin-S [43] network architectures, replacing their attention mechanisms with linear attention mechanisms. As shown in Tab. 8, our RALA significantly outperforms other linear attention mechanisms and also exceeds the performance of Softmax attention.

4.6. Ablation Study

Augmentation of the KV buffer. We first ablate the effect of augmenting the KV buffer on the model. As shown in Tab. 9, augmenting the KV buffer significantly increases its rank and enhances feature diversity, thereby leading to substantial improvements in model performance. This enhancement is particularly evident in downstream tasks such as object detection.

Augmentation of the output features. Since we use a linear projection to serve as $\phi(.)$, ablating the augmentation of the output feature reduces the model's parameter count.

Linear Attention	Params(M)	FLOPs(G)	Top1-acc(%)
Comparison on DeiT-T Setting			
DeiT-T [53]	6	1.1	72.2
Hydra Attn [1]	6	1.1	68.3
Efficient Attn [50]	6	1.1	70.2
Linear Angular Attn [67]	6	1.1	70.8
Enhanced Linear Attn [2]	6	1.1	72.9
Focused Linear Attn [23]	6	1.1	74.1
Ours	6	1.1	**75.1**
Comparison on Swin-T Setting			
Swin-T [43]	29	4.5	81.3
Hydra Attn [1]	29	4.5	80.7
Efficient Attn [50]	29	4.5	81.0
Linear Angular Attn [67]	29	4.5	79.4
Enhanced Linear Attn [2]	29	4.5	81.8
Focused Linear Attn [23]	29	4.5	82.1
Ours	29	4.5	**83.4**
Comparison on Swin-S Setting			
Swin-S [43]	50	8.7	83.0
Focused Linear Attn [23]	51	8.7	83.5
Ours	50	8.7	**85.0**

Table 8. Comparison of different linear attention based on DeiT-T, Swin-T, and Swin-S. RALA surpasses others by a large margin.

To ensure a fair comparison, we increase the expansion ratio of the FFN to maintain the model's parameter count and FLOPs at the same level. As shown in Tab. 9, the introduction of augmentation effectively increases the rank of the output features, thereby enhancing model's performance.

Conditional Position Encoding. We use CPE [6] to provide the positional information to our model. As shown in Tab. 9, CPE also improves the model's performance to some extent across various tasks. This indicates that CPE provides effective positional information for RAVLT.

Selection of $\phi(.)$ and $\kappa(.)$. We compare the choices of functions $\phi(.)$ (linear projection, identity map, and tanh) and $\kappa(.)$ ($\mathrm{Elu}(.)+1$ and $\mathrm{ReLU}(.)$), and experiments show that the model's performance improves with the inclusion of $\phi(.)$, regardless of the specific choice of it. This conclusion also holds for $\kappa(.)$, as shown in Tab. 9. **This indicates that the performance improvements stem from our RALA's augmentation of matrix's rank, rather than the selection of specific functions**.

Rank analysis of different layers. Under the DeiT-T setting, we analyze the rank of different layers of our model (12 layers in total, with each layer's feature matrix having a shape of $N = 196$, $d = 64$, and a full rank of 64). As shown in Fig. 8, RALA effectively increases the rank of the model's output features, thereby enhancing its performance.

5. Conclusion

In this paper, we investigate the causes of performance degradation in linear attention and conclude that the low rank of its output features is the primary factor. To address this issue, we propose Rank-Augmented Linear Attention (RALA). This method enhances two critical computation steps in linear attention—the calculation of the KV buffer and the computation of the output features—by employing two transformations. These augmentations effectively increase the rank of the matrices, thereby mitigating the low-rank problem in linear attention. Building upon RALA, we develop the Rank-Augmented Vision Linear Transformer (RAVLT), a novel architecture based on linear attention. RAVLT outperforms many existing Transformers, demonstrating significant potential for practical applications.

	Params	FLOPs	Top1	AP^b	AP^m	mIoU
	15	2.4	82.8	47.3	42.7	47.9
w/o KV aug	15	2.4	82.1(-0.7)	43.7(-3.6)	39.0(-3.7)	43.6(-4.3)
w/o out aug	15	2.4	82.5(-0.3)	46.3(-1.0)	41.6(-1.1)	47.0(-0.9)
w/o CPE	15	2.4	82.7(-0.1)	47.0(-0.3)	42.5(-0.2)	47.6(-0.3)
ϕ: proj→ identity	15	2.4	82.8(-0.0)	47.2(-0.1)	42.5(-0.2)	48.1(+0.2)
ϕ: proj→ tanh	15	2.4	82.7(-0.1)	47.2(-0.1)	42.7(-0.0)	47.7(-0.2)
κ: ELU + 1 → ReLU	15	2.4	82.8(-0.0)	47.3(-0.0)	42.8(+0.1)	47.7(-0.2)
κ: ELU + 1 → Softmax	15	2.4	82.7(-0.1)	47.1(-0.2)	42.7(+0.0)	47.8(-0.1)

Table 9. Ablation on RAVLT-T. The results show that performance improvements stem from our RALA's augmentation of matrix's rank, rather than the selection of specific functions.

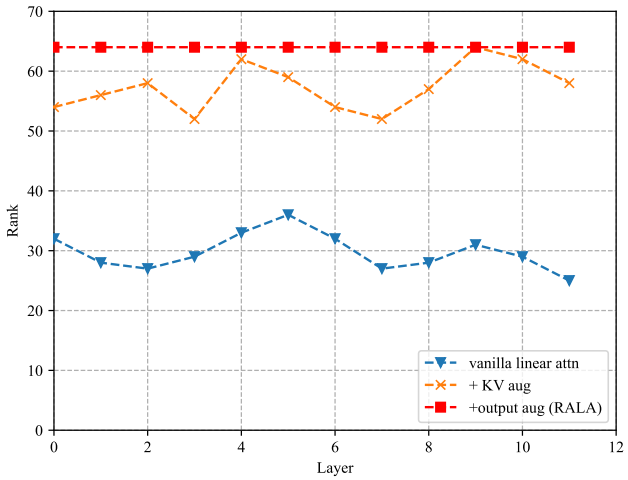

Figure 8. Analysis of the rank across different model layers. RALA's augmentation of the KV buffer and output feature effectively increases the rank of the output features at each layer.

6. Acknowledgements

This work is partially funded by Beijing Natural Science Foundation (4252054), Youth Innovation Promotion Association CAS(Grant No.2022132), Beijing Nova Program(20230484276), and CCF-Kuaishou Large Model Explorer Fund (NO. CCF-KuaiShou 2024005).

References

[1] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, and Judy Hoffman. Hydra attention: Efficient attention with many heads, 2022. 8

[2] Han Cai, Junyan Li, Muyan Hu, Chuang Gan, and Song Han. Efficientvit: Lightweight multi-scale attention for high-resolution dense prediction. In *ICCV*, pages 17302–17313, 2023. 1, 2, 8

[3] Zhaowei Cai and Nuno Vasconcelos. Cascade r-cnn: Delving into high quality object detection. In *CVPR*, 2018. 6

[4] Chun-Fu (Richard) Chen, Rameswar Panda, and Quanfu Fan. RegionViT: Regional-to-Local Attention for Vision Transformers. In *ICLR*, 2022. 3

[5] Kai Chen, Jiaqi Wang, Jiangmiao Pang, et al. MMDetection: Open mmlab detection toolbox and benchmark. *arXiv preprint arXiv:1906.07155*, 2019. 6

[6] Xiangxiang Chu, Zhi Tian, Bo Zhang, Xinlong Wang, and Chunhua Shen. Conditional positional encodings for vision transformers. In *ICLR*, 2023. 5, 8

[7] MMSegmentation Contributors. Mmsegmentation, an open source semantic segmentation toolbox, 2020. 7

[8] Zihang Dai, Hanxiao Liu, Quoc V Le, and Mingxing Tan. Coatnet: Marrying convolution and attention for all data sizes. *arXiv preprint arXiv:2106.04803*, 2021. 3

[9] Jia Deng, Wei Dong, Richard Socher, et al. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009. 5

[10] Mingyu Ding, Bin Xiao, Noel Codella, et al. Davit: Dual attention vision transformers. In *ECCV*, 2022. 2, 6

[11] Xiaoyi Dong, Jianmin Bao, Dongdong Chen, et al. Cswin transformer: A general vision transformer backbone with cross-shaped windows. In *CVPR*, 2022. 1, 2, 3, 6, 7

[12] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. 1, 2, 3

[13] Alaaeldin El-Nouby, Hugo Touvron, Mathilde Caron, Piotr Bojanowski, Matthijs Douze, Armand Joulin, Ivan Laptev, Natalia Neverova, Gabriel Synnaeve, Jakob Verbeek, et al. Xcit: Cross-covariance image transformers. *arXiv preprint arXiv:2106.09681*, 2021. 5

[14] Qihang Fan, Huaibo Huang, Jiyang Guan, and Ran He. Rethinking local perception in lightweight vision transformer, 2023. 2, 3

[15] Qihang Fan, Huaibo Huang, Xiaoqiang Zhou, and Ran He. Lightweight vision transformer with bidirectional interaction. In *NeurIPS*, 2023. 1, 3, 6

[16] Qihang Fan, Huaibo Huang, Mingrui Chen, and Ran He. Semantic equitable clustering: A simple, fast and effective strategy for vision transformer, 2024. 1, 2

[17] Qihang Fan, Huaibo Huang, Mingrui Chen, Hongmin Liu, and Ran He. Rmt: Retentive networks meet vision transformers. In *CVPR*, 2024. 2, 3, 5, 6, 7

[18] Li Gao, Dong Nie, Bo Li, and Xiaofeng Ren. Doubly-fused vit: Fuse information from vision transformer doubly with local representation. In *ECCV*, 2022. 2

[19] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. 3

[20] SG-Former: Self guided Transformer with Evolving Token Reallocation. Sucheng ren, xingyi yang, songhua liu, xinchao wang. In *ICCV*, 2023. 6, 7

[21] Jianyuan Guo, Kai Han, Han Wu, Chang Xu, Yehui Tang, Chunjing Xu, and Yunhe Wang. Cmt: Convolutional neural networks meet vision transformers. In *CVPR*, 2022. 2, 3, 7

[22] Meng-Hao Guo, Cheng-Ze Lu, Zheng-Ning Liu, Ming-Ming Cheng, and Shi-Min Hu. Visual attention network. *arXiv preprint arXiv:2202.09741*, 2022. 3, 6, 7

[23] Dongchen Han, Xuran Pan, Yizeng Han, Shiji Song, and Gao Huang. Flatten transformer: Vision transformer using focused linear attention. In *ICCV*, 2023. 1, 2, 3, 4, 6, 7, 8

[24] Dongchen Han, Ziyi Wang, Zhuofan Xia, Yizeng Han, Yifan Pu, Chunjiang Ge, Jun Song, Shiji Song, Bo Zheng, and Gao Huang. Demystify mamba in vision: A linear attention perspective. In *NeurIPS*, 2024. 1, 3, 5, 6, 7

[25] Ali Hassani, Steven Walton, Jiachen Li, Shen Li, and Humphrey Shi. Neighborhood attention transformer. In *CVPR*, 2023. 2, 6

[26] Ali Hatamizadeh, Hongxu Yin, Greg Heinrich, Jan Kautz, and Pavlo Molchanov. Global context vision transformers. In *ICML*, 2023. 1, 6

[27] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross B. Girshick. Mask r-cnn. In *ICCV*, 2017. 6

[28] Qibin Hou, Cheng-Ze Lu, Ming-Ming Cheng, and Jiashi Feng. Conv2former: A simple transformer-style convnet for visual recognition. *arXiv preprint arXiv:2211.11943*, 2022. 3, 6

[29] Gao Huang, Yu Sun, and Zhuang Liu. Deep networks with stochastic depth. In *ECCV*, 2016. 5

[30] Huaibo Huang, Xiaoqiang Zhou, and Ran He. Orthogonal transformer: An efficient vision transformer backbone with token orthogonalization. In *NeurIPS*, 2022. 2

[31] Huaibo Huang, Xiaoqiang Zhou, Jie Cao, Ran He, and Tieniu Tan. Vision transformer with super token sampling. In *CVPR*, 2023. 3, 5, 6, 7

[32] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Franc¸ois Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *ICML*, 2020. 1

[33] Manjin Kim, Paul Hongsuck Seo, Cordelia Schmid, and Minsu Cho. Learning correlation structures for vision transformers. In *CVPR*, 2024. 3, 6, 7

[34] Alexander Kirillov, Ross Girshick, Kaiming He, and Piotr Dollár. Panoptic feature pyramid networks. In *CVPR*, 2019. 7

[35] Youngwan Lee, Jonghee Kim, Jeffrey Willette, and Sung Ju Hwang. Mpvit: Multi-path vision transformer for dense prediction. In *CVPR*, 2022. 6

[36] Kunchang Li, Yali Wang, Peng Gao, Guanglu Song, Yu Liu, Hongsheng Li, and Yu Qiao. Uniformer: Unified transformer for efficient spatiotemporal representation learning, 2022. 6

[37] Siyuan Li, Zedong Wang, Zicheng Liu, Cheng Tan, Haitao Lin, Di Wu, Zhiyuan Chen, Jiangbin Zheng, and Stan Z. Li. Moganet: Multi-order gated aggregation network. In *ICLR*, 2024. 6, 7

[38] Yehao Li, Ting Yao, Yingwei Pan, and Tao Mei. Contextual transformer networks for visual recognition. *TPAMI*, 2022. 3

[39] Youwei Liang, Chongjian Ge, Zhan Tong, Yibing Song, Jue Wang, and Pengtao Xie. Not all patches are what you need: Expediting vision transformers via token reorganizations. In *ICLR*, 2022. 4

[40] Tsung-Yi Lin, Priya Goyal, Ross B. Girshick, and Kaiming He andPiotr Dollár. Focal loss for dense object detection. In *ICCV*, 2017. 6

[41] Weifeng Lin, Ziheng Wu, Jiayu Chen, Jun Huang, and Lianwen Jin. Scale-aware modulation meet transformer. In *ICCV*, 2023. 5, 6, 7

[42] Kai Liu, Tianyi Wu, Cong Liu, and Guodong Guo. Dynamic group transformer: A general vision transformer backbone with dynamic group attention. In *IJCAI*, 2022. 1, 2

[43] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021. 1, 2, 3, 6, 7, 8

[44] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, et al. A convnet for the 2020s. In *CVPR*, 2022. 6

[45] Jiachen Lu, Li Zhang, Junge Zhang, Xiatian Zhu, Jianfeng Feng, and Tao Xiang. Softmax-free linear transformers. *IJCV*, 2024. 1, 3, 6

[46] Jiachen Lu, Li Zhang, Junge Zhang, Xiatian Zhu, Jianfeng Feng, and Tao Xiang. Softmax-free linear transformers. *IJCV*, 2024. 7

[47] Yongming Rao, Wenliang Zhao, Yansong Tang, Jie Zhou, Ser-Lam Lim, and Jiwen Lu. Hornet: Efficient high-order spatial interactions with recursive gated convolutions. In *NeurIPS*, 2022. 3, 6

[48] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do imagenet classifiers generalize to imagenet?, 2019. 5

[49] Sucheng Ren, Daquan Zhou, Shengfeng He, Jiashi Feng, and Xinchao Wang. Shunted self-attention via multi-scale token aggregation. In *CVPR*, 2022. 2

[50] Zhuoran Shen, Mingyuan Zhang, Haiyu Zhao, Shuai Yi, and Hongsheng Li. Efficient attention: Attention with linear complexities. In *WACV*, 2021. 1, 2, 3, 4, 8

[51] Chenyang Si, Weihao Yu, Pan Zhou, Yichen Zhou, Xinchao Wang, and Shuicheng YAN. Inception transformer. In *NeurIPS*, 2022. 3

[52] Shitao Tang, Jiahui Zhang, Siyu Zhu, et al. Quadtree attention for vision transformers. In *ICLR*, 2022. 2

[53] Hugo Touvron, Matthieu Cord, Matthijs Douze, et al. Training data-efficient image transformers & distillation through attention. In *ICML*, 2021. 5, 7, 8

[54] Zhengzhong Tu, Hossein Talebi, Han Zhang, Feng Yang, Peyman Milanfar, Alan Bovik, and Yinxiao Li. Maxvit: Multi-axis vision transformer. In *ECCV*, 2022. 1, 2

[55] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *NeurIPS*, 2017. 1, 2, 3, 5

[56] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *ICCV*, 2021. 1, 3, 6, 7

[57] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pvtv2: Improved baselines with pyramid vision transformer. *Computational Visual Media*, 8(3):1–10, 2022. 1, 3, 7

[58] Wenxiao Wang, Lu Yao, Long Chen, Binbin Lin, Deng Cai, Xiaofei He, and Wei Liu. Crossformer: A versatile vision transformer hinging on cross-scale attention. In *ICLR*, 2022. 1, 2

[59] Wenhai Wang, Jifeng Dai, Zhe Chen, Zhenhang Huang, Zhiqi Li, Xizhou Zhu, Xiaowei Hu, Tong Lu, Lewei Lu, Hongsheng Li, et al. Internimage: Exploring large-scale vision foundation models with deformable convolutions. In *CVPR*, 2023. 6, 7

[60] Haiping Wu, Bin Xiao, Noel Codella, Mengchen Liu, Xiyang Dai, Lu Yuan, and Lei Zhang. Cvt: Introducing convolutions to vision transformers. *arXiv preprint arXiv:2103.15808*, 2021. 2

[61] Zhuofan Xia, Xuran Pan, Shiji Song, Li Erran Li, and Gao Huang. Vision transformer with deformable attention. In *CVPR*, 2022. 2, 3, 6, 7

[62] Tete Xiao, Yingcheng Liu, Bolei Zhou, Yuning Jiang, and Jian Sun. Unified perceptual parsing for scene understanding. In *ECCV*, 2018. 7

[63] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *AAAI*, 2021. 3

[64] Chenglin Yang, Siyuan Qiao, Qihang Yu, et al. Moat: Alternating mobile convolution and attention brings strong vision models. In *ICLR*, 2023. 5, 6

[65] Jianwei Yang, Chunyuan Li, Pengchuan Zhang, Xiyang Dai, Bin Xiao, Lu Yuan, and Jianfeng Gao. Focal self-attention for local-global interactions in vision transformers. In *NeurIPS*, 2021. 2

[66] Jianwei Yang, Chunyuan Li, Xiyang Dai, and Jianfeng Gao. Focal modulation networks. In *NeurIPS*, 2022. 3

[67] Haoran You, Yunyang Xiong, Xiaoliang Dai, Bichen Wu, Peizhao Zhang, Haoqi Fan, Peter Vajda, and Yingyan Lin. Castling-vit: Compressing self-attention via switching towards linear-angular attention during vision transformer inference. In *CVPR*, 2023. 3, 8

[68] Pengchuan Zhang, Xiyang Dai, Jianwei Yang, Bin Xiao, Lu Yuan, Lei Zhang, and Jianfeng Gao. Multi-scale vision longformer: A new vision transformer for high-resolution image encoding. In *ICCV*, 2021. 2

[69] Lei Zhu, Xinjiang Wang, Zhanghan Ke, Wayne Zhang, and Rynson Lau. Biformer: Vision transformer with bi-level routing attention. In *CVPR*, 2023. 1, 5, 6